

Flask Fundamentals Notes

1. What is Flask?

Flask is a **web framework for Python**. It allows you to build web applications using Python code. When a user visits a URL in the browser, Flask decides what to show them.

2. Creating a Flask App

```
python

from flask import Flask
app = Flask(__name__)
```

- `Flask(__name__)` creates the web application object.
 - This object is the heart of the entire app — everything is attached to it.
-

3. Routes

A **route** connects a URL to a Python function.

```
python

@app.route("/")
def home():
    return "Hello World"
```

- `@app.route("/")` means: when the user visits `http://127.0.0.1:5000/`, run the function below it.
 - The function must return something — a string, or a rendered HTML template.
-

4. `render_template()`

Instead of returning plain text, we can return a full HTML page.

```
python
```

```
from flask import render_template

@app.route("/")
def student_profile():
    return render_template("profile.html")
```

- Flask **automatically looks inside a `templates/` folder** for HTML files.
- So `profile.html` must be placed inside a folder called `templates/`.

Project structure:

```
your_project/
|
├─ app.py
├─ templates/
│   └─ profile.html
```

5. Passing Data to Templates

You can send Python variables (strings, booleans, lists) into the HTML file.

```
python

@app.route("/")
def student_profile():
    return render_template("profile.html",
                           name="Abc",
                           is_topper=True,
                           subjects=["Maths", "IT", "Science"]
    )
```

- `name`, `is_topper`, `subjects` are now available inside `profile.html`.
- Flask acts as the **middleman** — it passes data from Python into the HTML template.

6. Jinja2 Templating

Jinja2 is a templating engine built into Flask. It allows you to use Python-like logic inside HTML files.

Three main Jinja2 tags:

Tag	Purpose	Example
<code>{{ }}</code>	Display a variable	<code>{{ name }}</code>
<code>{% %}</code>	Logic (if, for, etc.)	<code>{% if is_topper %}</code>
<code>{# #}</code>	Comments	<code>{# this is a note #}</code>

Displaying a variable:

```
html
<h1>Hello, {{ name }}!</h1>
```

Output: `Hello, Abc!`

If / Else condition:

```
html
{% if is_topper %}
    <p>You are a topper!</p>
{% else %}
    <p>You are not a topper.</p>
{% endif %}
```

For loop:

```
html
<ul>
    {% for subject in subjects %}
        <li>{{ subject }}</li>
    {% endfor %}
</ul>
```

Output:

```
- Maths
- IT
- Science
```

Important: The browser never sees `{{ }}` or `{% %}` tags. Jinja2 replaces all of them with actual values **before** sending HTML to the browser.

7. HTML Forms and POST Requests

An HTML form collects data from the user and sends it to Flask.

html

```
<form action="/submit" method="POST">
  <input type="text" name="username" placeholder="Enter name">
  <input type="submit" value="Submit">
</form>
```

- `action="/submit"` — sends data to the `/submit` route in Flask.
 - `method="POST"` — the data is sent securely in the request body.
 - `name="username"` — Flask uses this name to identify the field.
-

8. Receiving Form Data in Flask

python

```
from flask import request

@app.route("/submit", methods=["POST"])
def submit():
    user = request.form.get("username")
    return render_template("welcome.html", name=user)
```

- `request.form.get("username")` — reads the value the user typed in the form field named `username`.
 - `methods=["POST"]` — tells Flask this route accepts POST requests.
-

9. Database with Flask (Introduction)

In real applications, data is stored in a **database** instead of being hardcoded.

The full flow:

```
User fills form → clicks Submit
↓
HTML sends POST request to Flask
↓
Flask reads data using request.form.get()
↓
Flask stores data in database
db.session.add(data)
db.session.commit()
↓
Flask fetches data from database
↓
Jinja2 fills HTML template with that data
↓
Final HTML page sent to browser
```

- **POST flow** → User submits data → Flask saves to database
 - **GET flow** → User visits page → Flask reads from database → Jinja2 renders it
-

10. Full Example — Student Profile Website

app2.py

```
python

from flask import Flask, render_template, request, redirect, url_for, session, Response

app = Flask(__name__)

@app.route("/")
def student_profile():
    return render_template("profile.html",
                           name="Abc",
                           is_topper=True,
                           subjects=["Maths", "IT", "Science"])

if __name__ == '__main__':
    app.run(debug=True)
```

templates/profile.html

html

```
<html>
  <head>
    <title>Student Profile</title>
  </head>
  <body>
    <h1>Hello, {{ name }}!</h1>

    {% if is_topper %}
      <p>You are a topper!</p>
    {% else %}
      <p>You are not a topper.</p>
    {% endif %}

    <h3>Subjects:</h3>
    <ul>
      {% for subject in subjects %}
        <li>{{ subject }}</li>
      {% endfor %}
    </ul>
  </body>
</html>
```

How it works step by step:

1. User visits `http://127.0.0.1:5000/`
2. Flask runs `student_profile()` function
3. `render_template()` sends 3 variables to `profile.html`:
 - `name = "Abc"`
 - `is_topper = True`
 - `subjects = ["Maths", "IT", "Science"]`
4. Jinja2 finds all `{{ }}` and `{% %}` tags and replaces them
5. Since `is_topper` is `True`, the `if` block runs → shows "You are a topper!"
6. The `for` loop runs 3 times → creates 3 `<li>` items
7. Final clean HTML is sent to browser

Final output in browser:

```
Hello, Abc!  
You are a topper!  
  
Subjects:  
- Maths  
- IT  
- Science
```

11. Login System Example

`app.py`

```
python
```

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("p1.html")

@app.route("/submit", methods=["POST"])
def login():
    user = request.form.get("username")
    pwd = request.form.get("password")

    valid_users = {
        "admin": "password",
        "user1": "pass123",
        "user2": "abc123"
    }

    if user in valid_users and pwd == valid_users[user]:
        return render_template("welcome.html", username=user)
    else:
        return render_template("p1.html", error="Invalid credentials")

if __name__ == '__main__':
    app.run(debug=True)
```

How it works:

1. User visits (/) → sees login form (p1.html)
 2. User fills username and password → submits form
 3. Flask receives data via request.form.get()
 4. Flask checks credentials against valid_users dictionary
 5. If correct → renders welcome.html with the username
 6. If wrong → re-renders p1.html with an error message
-

12. Key Concepts Summary

Concept	What it does
<code>Flask(__name__)</code>	Creates the app
<code>@app.route("/")</code>	Maps a URL to a function
<code>render_template()</code>	Loads an HTML file from <code>templates/</code> folder
<code>request.form.get()</code>	Reads data submitted by a form
<code>{{ variable }}</code>	Displays a Python variable in HTML
<code>{% if %}</code> / <code>{% endif %}</code>	Conditional logic in HTML
<code>{% for %}</code> / <code>{% endfor %}</code>	Loop in HTML
<code>Jinja2</code>	Engine that processes <code>{{ }}</code> and <code>{% %}</code> tags
<code>methods=["POST"]</code>	Allows a route to accept form submissions
<code>debug=True</code>	Auto-restarts server on code changes (dev only)

13. Roles of Each Technology

Technology	Role
HTML	Structure of the page — what the user sees
Flask	Brain — handles logic, routing, database
Jinja2	Connector — puts Flask's data into HTML
Database	Memory — stores all real data permanently

14. What's Coming Next

- Flask-SQLAlchemy — proper database setup and queries
- `GET` vs `POST` in detail

- `redirect()` and `url_for()` — navigating between pages
- `session` — remembering logged-in users
- Flask-WTF — better form handling and validation
- Blueprints — organising large Flask projects